

TP N°4, 5 et 6 – R2.01

Ce TP va courir sur 3 semaines. Il a pour objectif d'utiliser l'héritage, le sous-typage, la redéfinition, les classes abstraites, les interfaces et accessoirement certains des types collection générique de Java, les types `HashMap` et `Map`.

Toutes les classes développées doivent :

- 1) **Être dotées d'une Javadoc.**
- 2) **Respecter le principe de programmation défensive** (vérification systématique des paramètres d'appel, copie défensive lorsque nécessaire).

Il n'est pas demandé de tests unitaires pour chaque classe, mais simplement d'enrichir au fil de l'eau le `main` de la classe `TestServer` fournit en annexe pour démontrer que chaque version de votre application fonctionne correctement.

Vous devrez rendre votre code (les `.java`) et les documentations (`.doc`) dans un zip sur Moodle à la fin de chaque semaine en respectant le format `TPX-Nom-prenom-groupe.zip`.

Il y aura plusieurs versions de la même application, **chaque semaine faite une copie de la précédente** pour la faire évoluer dans un nouveau répertoire.

TP4 : développement de la première version de l'application d'alerte

Description générale du service d'alerte

Le service à développer permet à des utilisateurs (identifiés de manière unique par leur nom) de souscrire à un serveur de notifications qui peut les alerter en cas d'événements selon une modalité de communication qu'ils décident à l'inscription. Dans sa version initiale seulement deux modalités sont prévues : notification par « SMS » ou par « MAIL ». Mais les développeurs prévoient rapidement la mise à disposition de nouvelles modalités.

Architecture du service d'alerte

Ce service composé de 6 classes est entièrement contenu dans le package `notification`. Ce package respecte en l'état le *principe d'abstraction*. Il est conçu de manière à ne rendre public à ses utilisateurs que le strict nécessaire au fonctionnement du service : seule la classe `Server` est publique et donc visible depuis l'extérieur du package dans la version initiale du service.

Pour simplifier, on supposera que les applicatifs usant du service d'alerte sont simulés par la classe `TestServer` qui ne contiendra qu'un `main` démonstrateur du bon fonctionnement du code de ce service. Cette classe est située dans le package `application`. Le code de la version initiale de cette classe que vous serez amené à enrichir est donné en annexe.

Le code initial de la classe `TestServer` devra s'exécuter parfaitement avec la première version du service d'alerte que vous allez développer. Le résultat attendu de cette exécution est fourni également en annexe. La première version de ce service devra produire à l'exécution **exactement le même résultat**.

Architecture détaillée des 6 classes implantant le service d'alerte

Server

Seule classe publique du package pour l'instant, elle est le chef d'orchestre du service d'alerte.

Elle use du type `Map<String, Subscriber>` implanter concrètement via un `HashMap` (à importer tous les deux depuis `java.util`) pour typer son unique attribut `abonnements` et le concrétiser.

Un `Map<K, V>` en Java est une interface (donc un type pur sans code définissant uniquement des signatures de méthodes) qui associe des clés (type `K` ici `String`) à des valeurs (type `V` ici un type `Subscriber`). Elle se distingue ainsi d'une collection ordinaire : au lieu de stocker simplement une suite d'éléments, elle retient des paires clé-valeur. Vous n'aurez besoin pour coder `Server` que de :

- Retrouver la valeur associée à une clé grâce à `get(key)`
- Insérer de nouvelles associations avec `put(key, value)`
- Supprimer une association avec `remove(key)`.
- Récupérer par la méthode `values()` la colonne des valeurs sous la forme d'une collection de type `Collection<V>` à parcourir ensuite avec un `foreach`.
- Récupérer la taille de la Map avec la méthode `size()`.

Voici les méthodes de cette classe.

```
public Server()
```

Initialise l'attribut `abonnements` de type `Map<String, Subscriber>` avec une instance de `HashMap`.

```
public void adherer(String clientName, String mode, String adr)
```

Ajoute une souscription pour un nouveau client identifié par son nom et selon une modalité pour l'adresse fournie. Elle récupère depuis la classe `CommunicationFactory` une instance de communication de la bonne modalité, crée le nouveau `Subscriber` puis enregistre la souscription (une paire nom, subscriber) dans le Map. Pour l'instant un client ne peut souscrire qu'une seule fois au service et donc pour une seule modalité. Toute souscription supplémentaire est refusée.

```
public void retirer(String clientName)
```

Retire l'adhésion au service d'alerte pour le client cité s'il est adhérent, sinon affiche un message d'erreur.

```
public void alerter(String message)
```

Envoie le message passé en paramètre à l'ensemble des abonnées selon la modalité qu'ils ont choisie.

```
public String[] getListeInscrits()
```

Retourne sous la forme d'un tableau de chaînes la liste des abonnées selon le format pour chaque chaîne : « NomDuClient (modeChoisi) ».

Subscriber

Cette classe conserve les informations relatives à un abonné au service via 4 attributs.

- `String nom` : le nom du client (que l'on supposera unique parmi les abonnés).
- `String mode` : la modalité qu'il a choisie (par exemple une alerte par « SMS » ou par « MAIL »)
- `String adresse` : l'adresse dans la modalité choisie qui permettra l'envoi (par exemple

« 0631323334 »).

- `CommunicationStrategy strategy` : la modalité qui va servir à l'envoi des alertes

Voici les méthodes qu'elle propose.

```
public Subscriber(String nom, String mode, String adr,
CommunicationStrategy strategy)
```

Initialise un abonné avec son nom, le mode de communication qu'il a choisi pour l'adhésion, l'adresse pour le contacter dans la modalité choisie et l'objet de la bonne modalité qui sera en charge de l'envoi vers lui des alertes.

```
public String getNom() : Accesseur pour récupérer le nom du client.
```

```
public String getMode() : Accesseur pour récupérer le mode d'alerte du client.
```

```
public String getAdresse() : Accesseur pour récupérer l'adresse de contact du client.
```

```
public CommunicationStrategy getStrategy() : Accesseur pour récupérer l'objet en
charge de la communication dans la modalité qui a été choisie.
```

CommunicationFactory

Cette classe a la responsabilité d'instancier la bonne version de l'objet en charge d'une transmission selon une certaine modalité. Elle le fait via une unique méthode de classe. On ne doit donc pas pouvoir l'instancier (vous pouvez déclarer un unique constructeur sans paramètre `private` ne faisant rien pour le garantir !).

```
public static CommunicationStrategy create(String mode)
```

En fonction de la modalité reçue (actuellement uniquement « SMS » ou « MAIL »), elle décide de la création de la bonne instance d'une des sous-classes de la classe `CommunicationStrategy` à instancier et à retourner : donc soit une instance de `MailCommunication` si le mode est « MAIL », sinon une instance de `SmsCommunication` si le mode est « SMS ».

CommunicationStrategy

C'est la super-classe *abstraite* (donc non instanciable) décrivant l'interface publique de toutes les modalités de communication (pour l'instant il n'y en a que deux). Elle n'offre qu'une seule méthode *abstraite* (donc sans code) dont les implémentations concrètes dans les sous-classes permettront d'envoyer à un client identifié par son nom, à son adresse, un message dans la bonne modalité.

```
public abstract void envoyer(String clientName, String adresse,
String message)
```

MailCommunication

C'est une sous-classe *concrète* de la classe abstraite `CommunicationStrategy`. Elle redéfinit la méthode *envoyer* pour la modalité « MAIL ». Cette méthode produit un simple affichage tel que celui montré en annexe du sujet de TP comme résultat d'exécution du `main`.

```
public void envoyer(String clientName, String adresse, String
message)
```

SmsCommunication

C'est la seconde sous-classe *concrète* de `CommunicationStrategy` redéfinissant, elle aussi, la méthode *envoyer* pour la modalité « SMS ». Cette méthode produit un simple affichage tel que celui montré en annexe dans le résultat d'exécution du `main`.

```
public void envoyer(String clientName, String adresse, String message)
```

A faire pour le TP4

- 1) Dessinez pour vous un *diagramme de classes* de l'ensemble de l'application comportant les 7 classes et les 2 packages impliqués.
- 2) Développez impérativement dans l'ordre qui suit les classes de ce service. A chaque fois qu'une classe est écrite, compilez là.
 1. `CommunicationStrategy`
 2. `MailCommunication`
 3. `SmsCommunication`
 4. `CommunicationFactory`
 5. `Subscriber`
 6. `Server`
- 3) Compilez et exécutez le `main` de la classe `TestServer`. Vérifiez que tout fonctionne correctement et en particulier que l'affichage que vous obtenez est **rigoureusement conforme à celui donné en annexe**. Il est interdit de modifier pour cette version le `main` fourni.
- 4) Déposez l'intégralité de vos codes et leur documentation sur Moodle.

TP5 : seconde version du service d'alerte

Vous devez travailler sur une copie de la version de l'application de la semaine passée et cela dans un nouveau répertoire TP5. Chacune des évolutions décrites se rajoute à la précédente.

Première évolution : maintenance fonctionnelle => ajout d'une troisième modalité

Le service évolue on veut rajouter une troisième modalité de communication « XMESS » pour contacter les gens via un réseau social avec des adresses comme par exemple « @AliceTech ».

- 1) Identifiez les classes à créer et celles impactées par cette évolution. Dessinez le diagramme de classes du futur service. Quels choix de conception permet de rendre cette évolution simple à réaliser ?
- 2) Codez cette évolution.
- 3) Vérifiez, **en étendant le code du `main` de `TestServer`**, que cette modification est effective.

Seconde évolution : maintenance préventive => introduction d'une interface

Un développeur trouve que l'usage d'une classe abstraite `CommunicationStrategy` qui ne comporte aucun code concret héritable n'a pas d'intérêt : cela limite le potentiel d'usage de classes relevant d'autres branches de l'arbre d'héritage répondant pourtant aux besoins de communication du

service.

Il propose donc de supprimer la classe `CommunicationStrategy` et de la remplacer par une interface de même nom déclarant la même méthode.

- 1) Identifiez toutes les classes impactées par cette modification. Modifiez en conséquence le diagramme de classes de l'application.
- 2) Codez cette évolution dans le service d'alerte.
- 3) Vérifiez **sans modifier le code du main** de `TestServer` que cette modification est bien effective.

Troisième évolution : maintenance fonctionnelle imposant l'exploitation d'un code patrimonial

Un développeur déniché au sein du package utilitaire une classe de nom `ComNet` qui offre une nouvelle modalité « CHAT » d'envoi de messages. Malheureusement elle propose ce service avec une méthode qui a une signature différente de celle exigée par l'interface `CommunicationStrategy`. Sa signature est en effet : `public void send(String login, String message)`. Le login correspondant au paramètre « adresse » de la méthode `envoyer(...)`. Elle réalise cependant bien l'envoi souhaité mais ne fait aucun affichage manifestant cet envoi. L'équipe décide malgré tout de l'intégrer dans le service d'alerte via une classe `ComNetAdaptator` qui en réalise une *adaptation par composition*.

- 1) Développez dans le package utilitaire une nouvelle classe de nom `ComNet` qui comprend au moins la méthode `public void send(String login, String message)`. Pour vérifier son appel, trichez un peu et mettez lui une seule instruction : `System.out.print("*-ComNet ");`
- 2) Développez ensuite dans le package notification une nouvelle classe de nom `ComNetAdaptator` implémentant l'interface `CommunicationStrategy`. Elle a un unique attribut de type `ComNet` positionné par un constructeur sans paramètre. Elle offre une unique méthode `envoyer(...)` qui sous-traite dans son code à l'instance de `ComNet` de son attribut l'envoi du message puis réalise l'affichage que l'on attend habituellement.
- 3) Modifiez le reste du service d'alerte pour intégrer cette nouvelle modalité. Faites évoluer en conséquence le diagramme de classes de l'application.
- 4) Vérifiez en modifiant le code du main de `TestServer` que cette nouvelle modalité « CHAT » est désormais effective.

TP6 : troisième version du service d'alerte

Vous devez travailler sur une copie de votre version de la semaine passée et cela dans un nouveau répertoire TP6.

Première évolution : maintenance fonctionnelle => contrôle des adresses

Un développeur fait remarquer qu'il n'y a aucun contrôle syntaxique de la cohérence des adresses fournies selon chaque modalité de communication. Par exemple la présence d'un « 06 » ou « 07 » en entête suivi de 8 chiffres pour la modalité « SMS » serait une vérification minimale à réaliser. A

défaut, on s'expose à des crashes du service d'alerte lors de la tentative d'envoi de messages à des adresses incohérentes.

Il propose de rajouter à l'interface `CommunicationStrategy` une seconde méthode de signature : `public boolean isCorrect(String adresse)` qui fait quelques vérifications syntaxiques de base sur les adresses passées dans une modalité. L'idée est ensuite de vérifier lors d'une inscription dans la classe `Server` via cette méthode si l'adresse fournie est cohérente. L'abonnement n'est créé que si elle l'est sinon une erreur est affichée.

- 1) Intégrez cette modification au service d'alerte. Vous pouvez ne faire une vraie vérification que pour une seule des modalités (par exemple « SMS ») et pour les autres retournez systématiquement la valeur `true` en proposant une méthode « par défaut » dans l'interface (notez l'utilité ici d'user de cette possibilité pour éviter de coder cette méthode dans des classes implémenteuses préexistantes).
- 2) Vérifiez soigneusement en modifiant le code du `main` de `TestServer` que cette nouvelle évolution est opérationnelle.
- 3) Faites évoluer en conséquence le diagramme de classes de l'application.

Seconde évolution : maintenance perfective => éviter les objets inutiles et centraliser leur accès

Un développeur fait remarquer que l'on crée autant d'objets adressant une modalité de communication qu'il y a d'abonnés souhaitant être alerté selon cette modalité. C'est totalement inutile. Il faudrait un seul objet par modalité ; objet partagé par tous les abonnés relevant de cette même modalité. On évite ainsi la surcharge mémoire.

Plus encore, en ne permettant l'accès à ces instances depuis un seul endroit on facilite, pour plus tard, la possibilité de réaliser une évolution à l'exécution d'une modalité par simple substitution de l'objet qui en est responsable par un objet sous-type.

Nous allons donc non seulement garantir l'unicité des instances de communication mais également imposer une unique voie d'accès vers elles.

L'idée retenue est de ne pas toucher au code de la Classe `CommunicationFactory`. Ce code reste en l'état.

L'unicité pour chaque objet de type `CommunicationStrategy` à utiliser sera garantie par la classe `Server` : lors d'une demande d'adhésion elle ne demande à `CommunicationFactory` la création d'une instance de `CommunicationStrategy` pour une modalité qu'à la seule condition qu'elle n'existe pas déjà.

La centralisation de l'accès à toutes ces instances est obtenue en :

- supprimant la mémorisation de ces objets dans la classe `Subscriber`. Désormais cette classe n'a plus que trois attributs (le nom, la modalité et l'adresse).
- introduisant une nouvelle classe ayant uniquement des attributs et des méthodes `static` : `CommunicationAccess` va se consacrer au stockage et à la mise à disposition, pour le `Server` uniquement, dans un `Map<String, CommunicationStrategy>` d'une unique instance pour chaque modalité.

Cette nouvelle classe `CommunicationAccess` offre trois méthodes :

```
public static void setCom(String mode, CommunicationStrategy comObj )
```

Permet au `Server` de stocker dans la `Map`, l'objet en charge de la communication pour la modalité souhaitée. Si l'objet n'existe pas encore, la paire est ajoutée, sinon la nouvelle paire écrase la paire existante (inutile en l'état mais cela permettra plus tard de l'évolution dynamique si on veut la mettre en oeuvre).

```
public static CommunicationStrategy getCom(String mode)
```

Cette méthode retourne l'instance permettant l'accès selon la modalité passée en paramètre. Cette instance est celle contenue dans la `Map`. Si aucune instance n'est présente pour cette modalité la valeur `null` est retournée.

Un constructeur privé sans paramètre pour empêcher toute création d'instance de cette classe.

- 1) Créez la classe `CommunicationAccess`.
- 2) Modifiez le code la classe `Subscriber` pour retirer les objets instances de `CommunicationStrategy`.
- 3) Modifiez le code de la classe `Server` pour garantir l'unicité de création et le stockage et l'accès centralisé aux instances de `CommunicationStrategy` via la classe `CommunicationAccess`.
- 4) Sans aucune modification du code du main de `TestServer`, vérifiez que cette nouvelle maintenance est effective.
- 5) Modifiez le diagramme de classes de l'application.

Annexe

Code initial de la classe TestServer

```
package application;
import notification.Server;
/**
 * Cette classe ne contient qu'un main démonstrateur du bon fonctionnement
 * du service d'alerte.
 */
public class TestServer{
    /**
     * Exécution d'exemples d'usage du service.
     */
    public static void main(String[] args) {
        Server serveur = new Server();
        // Inscription
        serveur.adherer("Alice", "SMS", "0633343536");
        serveur.adherer("Bob", "MAIL", "bob@orange.fr");

        // Envoi d'une alerte
        serveur.alerter("Bonjour, voici une alerte importante!");

        // Désinscription de Bob
        serveur.retirer("Bob");

        // Ajout d'un nouveau client
        serveur.adherer("Charlie", "MAIL", "charlie@orange.fr");

        // Envoi d'une autre alerte
        serveur.alerter("Deuxième alerte!");

        // Affichage des inscrits
        String[] inscrits = serveur.getListeInscrits();
        System.out.println("Liste des inscrits : Nom Adresse (mode)");
        for(String ins : inscrits){
            System.out.println(ins);
        }
    }
}
```


Résultat de l'exécution du main initial de TestServer

[MAIL (bob@orange.fr) -> Bob] Bonjour, voici une alerte importante!

[SMS (0633343536) -> Alice] Bonjour, voici une alerte importante!

[SMS (0633343536) -> Alice] Deuxième alerte!

[MAIL (charlie@orange.fr) -> Charlie] Deuxième alerte!

Liste des inscrits : Nom Adresse (mode)

Alice 0633343536 (SMS)

Charlie charlie@orange.fr (MAIL)